
AAVARAN

On-Device Visual Perception for Light-Weight Browser Agents

An agent that reads the screen without the screen leaving the machine

Problem Statement **SIH26171**

Department of Space (ISRO) · Software · Smart Automation
Smart India Hackathon 2026

Submitted for team nomination by **Team Vagabonds**
Indian Institute of Management Mumbai

Harsh Bajpai	Architecture and redaction. The trust boundary; the redaction engine, covering field classification, the vault and the global sweep; the agent loop, and validation of every action the model returns
Jinshri Jain	Ledger design. How the Privacy Ledger presents itself: the on-screen against transmitted layout in Figure 2 that makes a redaction legible to a reader who is not an engineer, and the visual language carried into the deck
Vansh Khosla	Extension and integration. Build pipeline and manifests for Chrome and Firefox, the offscreen capture and masking path, continuous integration on three operating systems, and the one-command setup
Aarna Chauhan	Deck and diagrams. The submission deck and its figures, the architecture and trust boundary diagrams including Figure 1, and the layout and typography of this report
Manas Bharadia	Evaluation. The fixture corpus and its ground truth, the scorecard and the four measures in Table 1, the held-out discipline, and the adversarial and prompt-injection pages
Siddhartha Chaudhary	Demonstration. Its script and runbook, the onboarding material, and the content of the Indian checkout, banking and government pages the corpus is built from

17 September 2026

Abstract

A browser agent is only useful if it can see the page. That is also the reason people will not run one. The screen of an Indian user filling a form holds a PAN, an Aadhaar number, a card number, a billing address and an open password field, and every assistant of this kind in wide use today answers the problem by uploading a screenshot and asking to be trusted.

So we built one that does not have to be trusted. **Aavaran** runs a small model inside the browser, reads the page, and replaces every personal value with a typed tag such as `<PII_PAN_1>` **before any network request is made**. A larger open-weight vision-language model then reasons over the censored page and returns a single action, which the extension validates and executes. The tag carries the one fact the reasoning actually needs, namely that the field is filled and well-formed, and withholds the value itself. The secret store lives in the page context; the part of the extension that touches the network has **never held a real value** at any point in its execution, so a bug in that layer cannot leak a PAN. **The guarantee is a property of where the data sits, not a promise about code quality.**

It runs today on Chrome and Firefox, across Windows, macOS and Linux. Against a held-out set never used during development it redacts **every personal value with no false positives and leaks nothing**. A real multi-step task, filling and submitting a three-field form, finishes in a **median 30 seconds** and completes reliably; browser heap grows by **50 MB**; a turn puts **47 KB** on the wire. None of it needs a network.

We also report a weaker number on purpose, since it is the one that tells you something. Build a third corpus out of real captured pages, where the markup is nobody's doing but the site's, and **recall falls to 75%**. Anything below the extractor's node budget is simply never looked at. Chasing that figure turned up a sixth instance of the leak this project keeps rediscovering, which Section 5.5 describes. Section 4 sets out how the evaluation works before showing what it produced, and Section 7 says plainly what we have not proved.

Keywords. *on-device inference; privacy-preserving agents; PII redaction; vision-language models; browser extensions; WebGPU; held-out evaluation.*

1. The problem

ISRO's statement [1] asks for on-device visual perception for light-weight browser agents. Two constraints in that sentence do most of the work. *On-device* rules out the ordinary architecture, in which the page is shipped to a server that can see everything. *Light-weight* rules out the escape hatch of running a frontier model locally, because the target is a machine a student or a clerk actually owns.

The statement is worth reading for who floated it. A space agency runs restricted and air-gapped networks, and its people work on consoles holding material that cannot be handed to a commercial assistant. ISRO's own framing is blunt about why. Server-side pipelines limit what a user can share at all. A local agent removes the limit. The interesting part is that the same constraint binds a citizen filling an Aadhaar-linked form, which is why the fixtures in Section 4 are Indian retail, banking and government pages. A system that satisfies an organisation with genuine secrecy requirements is the same system that protects somebody's PAN, and we would rather demonstrate it on the harder of the two.

1.1 Why the obvious solutions fail

The first instinct is to blur the screenshot. It does not survive contact with a real page, because an agent that cannot read the field cannot act on it either; blurring the PAN also blinds the agent to whether the PAN box is filled, which is the thing it needs to know in order to decide what to click next.

The second instinct is to redact on the server, immediately on arrival. This concedes the entire point. The value has already crossed the network by the time it is redacted, and the user is once again being asked to trust an operator and a log retention policy rather than an architecture.

The third is to run everything locally, which is correct in principle and is roughly what we do, but only becomes a real answer once someone has measured what a seven billion parameter vision model costs on a laptop with no discrete GPU. Much of our engineering time went into that measurement rather than into the idea.

1.2 Related work

PII detection itself is not new. Presidio [3], originally from Microsoft and now community-governed, combines named-entity recognition, pattern matching and checksums, and commercial equivalents exist in every major cloud. What they share is placement: they run where the data has already arrived. That is the right tool for a log pipeline and no help at all for the problem in Section 1, because by the time such a system sees a PAN it has crossed the network. Our contribution is not a better detector, and Table 2 shows our pattern layers alone are unremarkable. It is that detection is moved to the one place where refusing to send something is still possible.

The client-side half rests on work recent enough to make this feasible at all. WebGPU [5] and ONNX Runtime Web [6] put real inference inside a page, and Transformers.js [7] makes loading it practical, which is what the problem statement means when it points at them. For reasoning we use Qwen2.5-VL [2] served by Ollama [8] rather than a hosted API, so no part of the pipeline requires a network.

For Section 4.8 the relevant literature is indirect prompt injection [4], which established that content a model retrieves is an attack surface rather than inert data. We report our result in those terms, including the part of that threat our architecture does not address.

2. Approach

The design rests on one observation. An agent deciding what to do next does not need to know that the PAN field contains ABCPE1234F. It needs to know that the field labelled PAN contains a well-formed PAN. Those are different facts, and only the second has to leave the machine.

So the extension extracts a structured description of the page in the content script, classifies every value, and substitutes a tag that preserves the type and the validity while discarding the content. The real values are held in a vault that never leaves the page context. When the model replies with an action such as `click(e1_24)`, the extension resolves the reference locally and executes it. The model has directed an action against a value it was never shown.

Redaction is global to the vault rather than local to a node. If a value is recognised once, every later occurrence of the same string is swept, which is what stops a card number that also appears in a confirmation sentence from escaping through the prose.

2.1 Architecture

Figure 1 shows where the boundary sits. Everything to the left of the dashed line runs in the page and holds real values. Everything to the right has only ever seen tags.

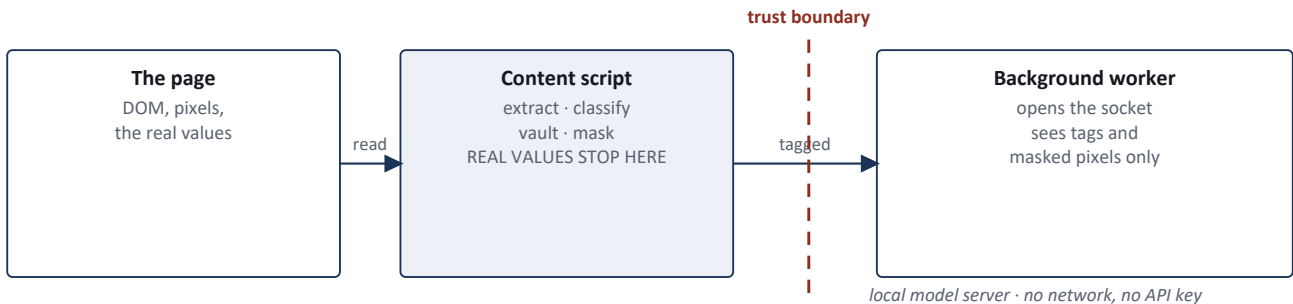


Figure 1. The trust boundary. The vault and the classifier run in the content script; the background worker, which is the only component that opens a socket, receives tagged text and masked pixels and nothing else.

3. The system as built

A turn runs in five stages. The page is extracted into a node tree, pruned of anything invisible or decorative. Elements that carry meaning only as pixels, such as a canvas or an image with no alternative text, are queued for the in-browser vision model. Every text value is classified and, where personal, replaced by a tag. A screenshot is captured, and every region that was redacted in the text is filled solid in the image before it is encoded. The censored bundle goes to the local model server, and the single action that comes back is validated against the node tree before it is allowed to run.

The fourth of those stages exists because of a failure described in Section 5, and it is the one we had not thought to build until the failure in Section 5.1 made the case for it.

4. Evaluation

ISRO's statement sets out how entries will be scored, so this section reports against that rubric rather than against measures of our own choosing. The method is given before the numbers, and the numbers are given with their denominators.

4.1 The corpus, and what it is not

Eleven scored fixtures cover Indian checkout, banking, government and support pages, including forms in Hindi, Tamil and Bengali, pages using shadow DOM, and a page embedding a third-party frame. Three further pages exercise behaviour that is not scored as a page: hostile content, coordinate alignment, and a multi-step journey. Alongside these sit two held-out pages, and four captures of real public websites.

The held-out pages were written by us. They were authored before the first scoring run and have never been tuned against, which is a real discipline and the reason the tuned and held-out columns can be compared at all. But held out from tuning is not the same as drawn from an independent distribution, and we would rather say so than let the word do work it has not earned.

So there is a third corpus, and it exists precisely because of that objection. **The wild set takes the four real captured pages, keeps their DOM exactly as it is, and injects a small block of synthetic personal data with known ids.** The values and labels are ours, because ground truth on a stranger's page cannot be invented. Everything the extractor has to fight through is not: thousands of nodes, real class names, real nesting, whatever each site happens to do. That is the half that breaks things. A form in a hand-written thirty-node fixture and the same form buried in a six-thousand-node tree are not the same test, and the second is the one December will present. Twelve values, four decoys, four pages.

The rule we do enforce absolutely is that nothing may be special-cased to the held-out set. The grand finale reveals its own use cases on the day, so a system fitted to its own corpus predicts nothing about December. The same reasoning bans hardcoded CSS selectors anywhere in the codebase.

4.2 Results against the published rubric

ISRO metric	Weight	Tuned (11 pp.)	Held out (2 pp.)	Wild (4 pp.)
Accuracy of visual context	25%	100% (14/14)	100% (4/4)	75% (3/4)
PII recall	20%	100% (52)	100% (12)	75% (9/12)
PII precision	20%	100%	100%	100%
Precision of redaction	20%	100% (48/48)	100% (10/10)	100% (9/9)
Client resource use	20%	+49.5 MB heap	see 4.3	see 4.3
End-to-end task latency	15%	63 s median, vision included	see 4.3	see 4.3
Leaks (our own invariant)	n/a	0	0	0

Table 1. Produced by `bench/score.ts` on 17 September 2026. Denominators are given because a rate without one is not a measurement. Visual context accuracy counts expected interactive elements recovered from the page against a hand-written ground truth file per fixture; it is not a human judgement of ours.

Precision deserves a note, because it is where this system was once genuinely bad. An early version scored **22% redaction precision** on held-out pages: it was reading the cell `Applicant Name` on a government form and redacting the words “Applicant Name” as though they were somebody’s name. The rule that fixed it is that **evidence about a value must never be drawn from the value itself**, and that rule recurs throughout Section 5.

The last two rows carry 35% of the score between them, and they were for a long time the weakest thing in this report: a heap delta measured in a Node process, and a latency excluding both vision and the model. Neither is what ISRO asked for. Both are now measured where they exist, in a real browser driving a real task, and Section 4.3 describes that run.

4.3 The end-to-end task, measured in a browser

The rubric asks for latency of a *task*, not of a turn, so the measurement is a task. A grievance form is loaded in Chrome with the extension installed and the local model running, and the agent is given one instruction: choose a category, enter a reference number, write a description, submit. Submit is disabled until all three fields are filled, so this cannot be satisfied by a single lucky click.

It completes in a **median of 30 seconds** across six turns, and browser heap in the page grows by about **50 MB**. Success is not taken from the loop’s own report: the harness reads the page afterwards and requires every field populated and the confirmation banner actually visible. Section 5.4 explains why that distinction turned out to matter more than anything else in this report.

That figure excludes the vision stage, and saying so matters more than the number. The measurement page contains nothing the DOM cannot describe, so nothing ever enters the vision queue and every turn records zero milliseconds of vision. The thirty seconds is therefore the latency of the DOM path alone. We measured the difference rather than estimating it: on a variant of the same page carrying one image, the vision stage runs on all six turns and costs about seven seconds in total, and the same task then takes a median of sixty-three seconds, the balance being model time that varies with machine load. That variant is not the one used for the headline figure, because adding the image changed what the model saw each turn and made the task itself less reliable. Both numbers are true of the pages they were taken on, and a reader should assume a real page with images sits nearer the larger one.

This was found only because a crash in the vision stage forced the question of whether that stage had ever run during a benchmark at all. It had not. A measurement can be green, reproducible and still not execute the code it claims to time.

It is also not deterministic, and a single figure would hide that. Seven of those eight runs completed the task; one did not. The agent is a 7B model choosing its own actions, so the same instruction does not always produce the same sequence. When it fails it fails safely, leaving the form partly filled rather than submitting something wrong. But a demonstration that works seven times in eight is one that can fail in front of you, and making the loop recover reliably is honest work still to do.

4.4 What the perfect scores do and do not mean

Every cell in the tuned and held-out columns reads 100%, and we would treat that with suspicion in someone else's report, so we will do so in our own. Zero misses is not the same as a low miss rate. On the held-out set the denominator is **12 personal values across 2 pages**. By the rule of three [9], observing no failure in twelve trials is consistent with a true miss rate as high as **25%**. The tuned set is stronger at 52 values, giving an upper bound near 6%, but it is the set we developed against.

So the correct reading of the first two columns is that the system did not fail on any value it was shown, not that it fails rarely. **The wild column is where that distinction stops being theoretical.** Recall there is 75%, not 100%, and the three missed values are all on the same page: the Wikipedia capture is 5,993 nodes against a 1,500 node budget, so the injected block sat below the cap and was never extracted at all. Section 5.5 describes what that turned out to mean.

4.5 What the context layer is worth

A reasonable objection is that patterns and checksums alone might do most of this, and that the surrounding machinery buys little. The scorecard can be run with the contextual name layer disabled, which answers it directly.

Configuration	Held-out recall	Values leaked
Patterns and checksums only	91.7% (11/12)	1
Full pipeline	100% (12/12)	0

Table 2. Ablation via `score.ts --no-layer3`. On the tuned corpus the same ablation leaks five values rather than one.

In percentage terms the layer is worth eight points. In the terms that matter for this problem it is the difference between a system that leaks a name and one that does not, and a privacy guarantee is not an average.

4.6 Real websites, where it does worse

The four real captures carry no ground-truth labels, so recall cannot be scored on them. Precision can be, because these pages hold almost no personal data of the user: on Hacker News, python.org, an RBI circulars page and a Wikipedia article, nearly anything redacted is a false positive. **The system redacts 21 items across the four pages, and that number was 24 until this corpus was measured.**

Three of those were not names at all and have since been fixed. An RBI circulars page yielded “Master Directions” and “Master Circulars”, which are document types, and “Not Pressed”, which is a case status. The cause was that “Master” and “Not” are both genuine given names and ordinary English words, so a rule meant to catch an uncommon surname after a known given name dragged the following word in with it. Requiring that the trailing word not be an ordinary English word removed all three at no cost to recall, and that page now yields nothing.

So redaction precision on pages we did not write is materially below the 100% in Table 1. The failure mode is over-redaction rather than leaking, which is the safer direction, but the rubric weights precision of

redaction at 20% and an agent blinded to the names in a document is less useful. The cause is easy to state. A name in prose and a name in a form field are treated alike, when only the second belongs to the user. Fixing it means distinguishing the user's PII from third-party names in published content, which is a real piece of work and is now in Section 8. We report it here because a reader will otherwise find it in the first minute of trying the system.

4.7 The ledger

Because a privacy claim that cannot be checked is only a slogan, the extension ships a ledger that shows, for a given request, what was on the screen, what was transmitted in its place, and the exact bytes that went out. Figure 2 is the ledger for the checkout fixture, produced by the run that generated Table 1.

4.8 Adversarial behaviour, and its limit

We added a hostile page to the corpus. Hidden in it was an indirect prompt injection [4] telling the agent to disclose everything it knew. It fell for it completely, announcing that the user had authorised full disclosure. Then it **disclosed nothing**, because at that point in the pipeline it held no value to disclose. This is the clearest illustration of why the guarantee is structural: a behavioural defence has to win every time, an architectural one does not have to win at all.

The limit of that result should be stated, because it is easy to overclaim. What we showed is that injection cannot extract data. We did not show that injection cannot steer the agent's actions. The model did comply, and our validation checks that a returned action is well-formed and names a real element on the page; it does not check that the action is a sensible thing to do. An injected page that persuades the agent to click a genuine button it should not have clicked would pass. Action-level hijacking is the live risk in agent security and we have not tested it.

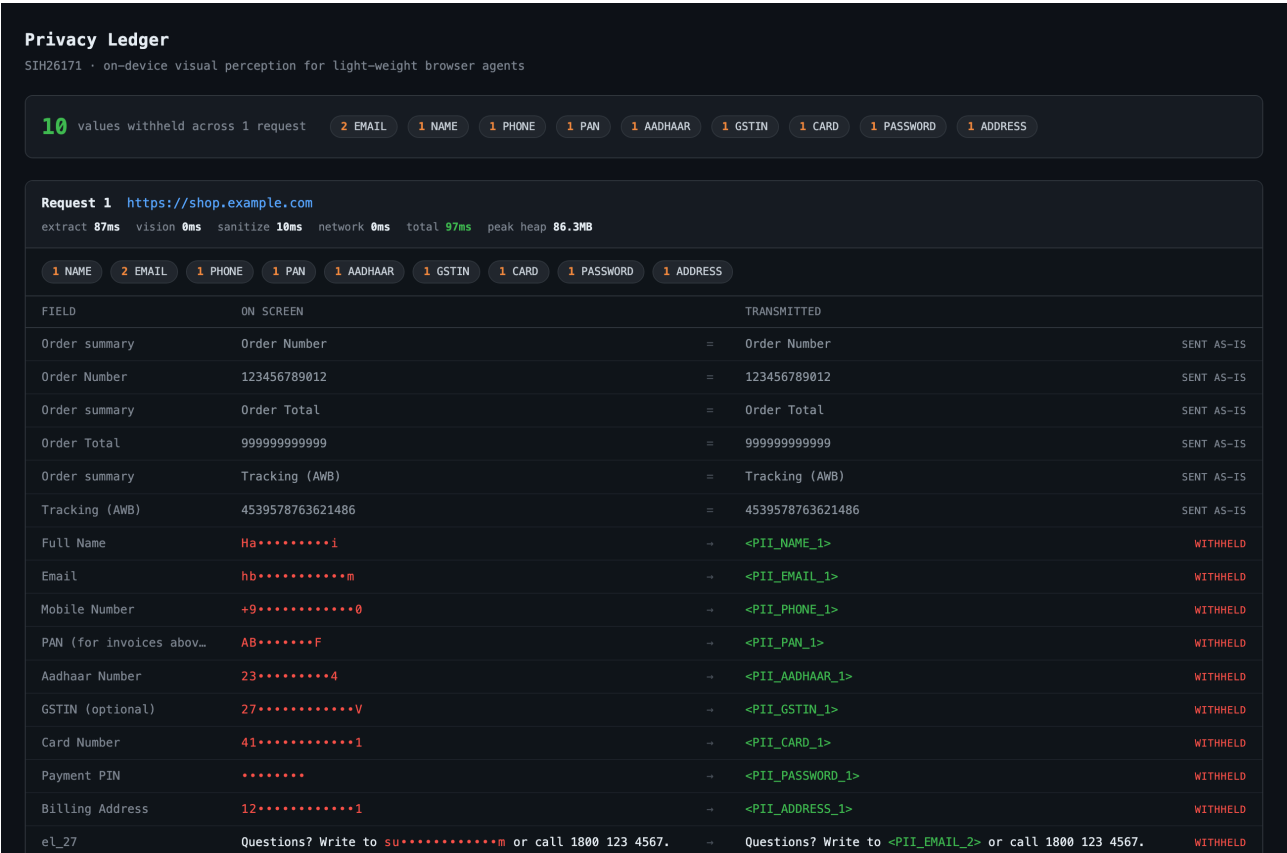


Figure 2. Ten values withheld from a single checkout page. The left column reconstructs what stood on screen from masked shadows, since storing the real values in order to display them would be worse than the problem the system solves. Order number, total and tracking number are correctly judged harmless and sent unchanged.

4.9 Threat model

The claim is bounded and worth bounding explicitly. We defend against the **server operator and the network**: the reasoning model, anyone hosting it, and anyone on the path sees tagged text and masked pixels, never a value. That holds even if the server is hostile, because it is a property of what is sent.

We do not defend against an attacker who already controls the machine. The extension necessarily holds permission to read pages, the vault lives in browser memory, and a malicious extension update or a compromised browser defeats the design completely. Nor does the architecture prevent the local model server from logging what it receives, though what it receives is already redacted. Stating this narrows the claim to one we can actually support.

Three boundaries are worth naming outright, because each is the first thing a reader tries. The system does not stop the user photographing their own screen: masking applies to the image this agent captures and transmits, while an operating system screenshot reads the framebuffer, which no browser extension can observe. It does not police other extensions or other agents; Chrome deliberately grants no extension authority over another's access to a page, and a product claiming that capability is either overstating it or is a different class of software altogether. And it does not hide anything from the website itself, which already holds the value because the user typed it there. The guarantee is about what leaves for a third party. Aavaran is a redaction boundary inside one agent, demonstrable byte by byte, and not a guard around the whole machine.

A fourth limit was found on a live page and is reported for the same reason. A site that renders its content inside a cross-origin frame is invisible to the content script, which runs in the top frame only. Until 18 September such a page was scanned, read almost nothing, and was reported as carrying nothing personal. A clean result issued while blind is the most dangerous output a privacy tool can produce, because the reader concludes the page is safe. The share of the viewport that could not be read is now measured, and a page we could not substantially read is reported as unreadable rather than clean.

4.10 Tests and continuous integration

Twenty-six checks run on every push, nineteen of which need no browser, across Linux, macOS and Windows. They cover unit behaviour, the two privacy invariants, the scorecard on both corpora, adversarial pages, deliberate server faults, and timing under a six-fold CPU throttle. The nineteen browser-free checks last ran green, with nothing skipped, on 18 September 2026.

5. What went wrong, and what it cost

We think this section is the honest measure of the project, so it is reported rather than summarised. The work was not a straight line from idea to demo.

5.1 The leak that kept coming back

Five separate defects turned out to share one shape: content that was visible on screen but invisible to the redactor, while the screenshot transmitted it anyway. Hidden text was lifted into a visible element's label. Shadow DOM was never traversed. Text beyond an internal character cap was silently truncated. A value appearing twice was redacted in one place only.

The fifth was the general case and the worst. A value could be perfectly tokenised in the JSON and perfectly legible in the picture sent in the same request, because the masking step had only ever been given face boxes and had never masked any text at all. On the demo page itself, **ten values were stripped from the payload while an image of all ten went out beside it**. The leak test had passed throughout, because it inspected the payload and had never once looked at the image.

A system that emits two artefacts has to be tested for agreement between them. Asserting on one proves nothing whatever about the other.

The repair fills each redacted region solid rather than blurring it. A blur radius tuned to destroy a face leaves text perfectly legible. It also fails closed. If fewer regions are masked than were asked for, the screenshot is withheld outright rather than sent clean. The same audit found that an embedded frame is a rectangle the content script cannot see into while the camera photographs it regardless, which is an ordinary arrangement on Indian banking and government portals, where the payment or KYC step is embedded from elsewhere.

5.2 Tests that could not fail

One check passed while the extractor was returning zero nodes, because in JavaScript every assertion over an empty list is vacuously true. Worse, the test written specifically to catch that class of mistake had the same defect: it read its expected count from the code under test, so sabotaging the code drove expectation and actual to zero together and the suite stayed green. Expectations are now derived from sources the implementation cannot move, and we verify a new check by breaking the fix deliberately and confirming the check goes red.

That discipline earned its keep on the morning this report was written. Preparing Figure 2, we noticed the ledger labelling a row with another row's data. The cause was a fix made six days earlier: reading a table's column headers from its first row, without first checking whether that row was a header row at all. In a table with no header, the first row is data, so every later row was being labelled with row one's values, and the order total's context read 123456789012 Order Total. That is the rule from Section 4.2 broken by the change written to enforce it. **The whole suite had passed before and after, which is to say nothing covered it.** The fix is one additional condition; the test guarding it fails in four places when that condition is removed.

5.3 Measurements that contradicted us

Nearly every performance intuition we had was wrong, and instrumenting always cost less than guessing. Quantising the in-browser vision model to int8 made it about ten times slower than fp32, because the WebGPU backend has no native int8 matrix path. Reading an element's children through `Array.from(el.children)` per node took 8,644 ms where sibling iteration took 4 ms. Resolving label elements with a document query per field took 54 seconds across 1,500 calls, against building one map at the start. Chrome's offscreen document is timer-throttled, so encoding even a sixteen-pixel canvas there takes about a second.

Two of these were wrong in the other direction, which is worth admitting. We once recorded that Firefox exposes no WebGPU adapter. That holds only for headless Firefox. A headed browser has one, and the original note still sits in the repository, marked superseded rather than quietly deleted. Separately, a timing test that asserted a fixed six-second ceiling failed two documentation-only commits at 6,021 ms on a loaded runner, because a wall-clock threshold is a claim about the machine rather than about the code. It now asserts a ratio against a reference operation timed on the same machine. Relaxing a guard means re-earning its teeth. So we broke the code on purpose and watched: the check fired at 3.40 times the limit.

5.4 The check that could not tell success from failure

The end-to-end agent loop is the thing a judge will actually watch, and it had the weakest check in the project. The harness launched Chrome, ran the task, wrote a result file, and reported success if that file existed. It never read what was in it.

So the suite reported the agent loop as passing through **every run in which the agent filled two fields of three and then gave up**. The loop had run. The task had failed. The check could not distinguish those, and nobody looked, because it was green.

Reading the page afterwards instead exposed three real defects in a row. The action schema allowed a type with no value, so the model emitted one and lost a turn to a refusal. A repair using JSON Schema `if/then`

was accepted by the runtime and silently ignored, giving a constraint that looked stricter without being stricter; oneOf is converted into a real grammar and was verified on both branches before being adopted.

The third was the one that mattered. Refused actions were never added to the history the model sees, so the model proposed an invalid action, never learned it had been rejected, and proposed it again; the loop then stopped, reasoning that another identical turn would only repeat the refusal. That was true, but only because the refusal was being withheld from the one participant who could act on it. **The assumption made itself true.** Putting the refusal into the history and allowing one re-plan is what made the task complete, and the measurement in Section 4.3 exists because of it.

5.5 The sixth instance, found on the first run of a new corpus

The wild corpus was built to answer a fair objection, that we had written our own exam. It found a leak within minutes of existing.

When a page exceeds the node budget the walk stops. Everything below the cap is invisible to the redactor and perfectly visible on screen, and the rules that decide whether to withhold the screenshot consulted the closed-shadow-root case and the text-cap case but **never consulted truncation at all**. So on any large real page, a PAN below the cap was absent from the payload and present in the transmitted image. That is the same shape as the five leaks in Section 5.1, for the sixth time, and it had been sitting behind three corpora that were all small enough to fit inside the budget.

Withholding on truncation alone would have been far too blunt, since almost every large page truncates and the screenshot carries 25% of the grade. The narrower question the text cap already asks is the right one: is there anything PII-shaped in the part we did not reach? Only then is the image unsafe.

The first version of that check scanned a bounded sample of the missed content. On Wikipedia the bound was exhausted by ordinary prose thousands of nodes before reaching the block that held the GSTIN, so **it reported clean on the exact page that motivated it**. A guard that runs out of budget before reaching the danger is worse than no guard, because it also reports success. It now sweeps form controls first and exits on the first verified hit.

6. Performance and deployment

A full turn transmits **47 KB**. In-browser vision takes 65 ms; the reasoning model takes about 3.8 seconds, giving roughly 4.7 seconds end to end on a laptop throttled to six times slower than the development machine. The model generates about 11 tokens per second, and that figure is hardware rather than a tuning failure: constrained decoding, context length and model choice were each measured separately and ruled out.

A task costs more than a turn, and the difference is not in our favour. The grievance task in Section 4.3 takes about half a minute over six turns, slower than filling the form by hand. The honest position is that this system currently earns its place where the value is privacy rather than speed. The 3.8 seconds of model time per turn is the whole of the gap, and it is already a quantised number: the model runs at Q4_K_M, so the obvious lever has been pulled. Two cheaper ones have not. Of the six turns that task took, **two were wasted** clicking a button that had already done its job, because the model would not emit the action that says it has finished. Ending the run when the page shows the task is done would remove about a third of the wall-clock time without touching the model at all.

The reasoning model is Qwen2.5-VL at 7B [2], open weights, served locally through Ollama [8] in about 6 GB. **Nothing in the pipeline requires a network connection or an API key**, which matters for the deploying organisation as much as for the individual user.

7. What we have not proved

Stating these plainly seems more useful than leaving them to be discovered.

The screenshot masking is now proven on pixels, and this paragraph used to say the opposite. A spike loads the checkout fixture in headed Chrome, runs the production capture and mask path, and samples the transmitted image. All ten redacted regions have their luma variance collapse to essentially zero, which is what a solid fill does and a blur does not, while control points away from every box stay byte-identical. The count of regions expected comes from the hand-written ground-truth file rather than from the sanitizer, so breaking the sanitizer cannot drive expectation and actual to zero together. Reverting the fill turns it red, and instructively so: the box count still reads ten of ten, and only the pixels give it away.

Redaction precision on pages we did not write is worse than Table 1 suggests. Section 4.6 gives the measurement: 21 items redacted across four real websites. Three of the clearest errors have since been fixed, but seventeen names from a single encyclopedia article remain, because the system does not separate the user's own personal data from third-party names in published text.

We have not shown that the agent cannot be steered. Section 4.8 sets out the distinction. Injection cannot extract a value from us; whether it can induce a bad action is untested.

Embedded frames are masked rather than read, so their contents are protected but their meaning is lost to the agent. Recovering it means merging separate vaults across frames, which is exactly where leaks of the Section 5.1 kind would live, so it needs a design rather than a flag. Eight of the eleven Indian-language vocabularies have not been reviewed by a native reader. Name detection in running prose is currently Latin-script only, though form fields work in every language we support.

8. Plan to the grand finale

The order is deliberate: close the one unproved claim, then widen the evidence, then extend the capability.

Priority	Work	Why it is in this order
1	Separate the user's own data from third-party names in prose	The largest measured defect; costs precision, worth 20% of the rubric
2	Extract below the node budget instead of withholding the image	Failing safe costs visual context, worth 25%
3	Grow the wild corpus to real government portals	24 labelled values cannot separate a good system from a lucky one
4	Native review of eight language vocabularies	Cheap, high value, and needs a reader rather than an engineer
5	Read inside frames by merging per-frame vaults	Recovers lost context, but only with a design that cannot leak
6	Stop the run when the task is visibly done	Two of six turns were wasted; the model is already 4-bit

9. The team

All six of us are students of the Indian Institute of Management Mumbai, which satisfies the requirement that a team may not be drawn from more than one college. Two members, **Jinshri Jain** and **Aarna Chauhan**, are women, so the **requirement of at least one female member is met**.

The work divides along the three parts of the system. **Harsh Bajpai** built the architecture and the redaction engine. That is where the guarantee in Section 2 is won or lost, and it is also where four of the five leaks in Section 5.1 were found and closed. The agent loop, which checks every action before it runs, is his too.

Around that sits the extension, and **Vansh Khosla** built it: the repository, the builds for two browsers that want different things, the capture and masking path described in Section 5.1. He also set up the integration

that runs the whole suite on Windows, macOS and Linux at every push. That last piece is why we can claim anything at all. A number nobody re-checked on another machine is a memory, not a measurement.

Manas Bharadia built the evaluation. Corpus, ground truth, the four measures in Table 1, and the held-out rule that stops us marking our own homework. The uncomfortable numbers in this report are his work as much as the flattering ones.

The remaining half decides whether any of it can be understood by someone who did not build it, and we would not claim it matters less. **Jinshri Jain** designed how the Privacy Ledger presents itself. Figure 2 is hers, and it is the page that turns a privacy claim into something a reader checks rather than accepts. **Aarna Chauhan** produced the submission deck, the diagrams including Figure 1, and how this report reads on the page. **Siddhartha Chaudhary** built the demonstration and its runbook, and owns the Indian-language coverage. A missing word for “total” in Gujarati costs us precision, not recall, which is an easy thing to overlook and worth 20% of the marks. The repository carries continuous integration on three operating systems and two tagged releases.

10. What we are asking for

We would be grateful to be considered for nomination by the institute against problem statement SIH26171. The portal records the last date for team nomination and idea submission as **30 September 2026**, and the statement currently holds 20 of the 500 ideas it will accept.

Should the team be taken forward, two things have to come from the institute. The nomination itself, and the team identifier the portal issues, which is printed on the submission deck. We would also welcome guidance on how you would like the internal selection handled, and we are glad to prepare whatever documentation, presentation or demonstration would be useful for it. If anything in this report needs expanding or evidencing further, we will gladly do that too.

We would be happy to demonstrate the system in person at your convenience, on a machine with no network connection, which is the most direct way to see that the claim in the abstract is a fact about the architecture rather than a statement of intent.

References

Sources are given for claims resting on someone else's work. Software with no accompanying paper is cited as a project rather than invented into one.

- [1] Indian Space Research Organisation. *On-device Visual Perception for Light-weight Browser Agents*. Smart India Hackathon 2026, Problem Statement SIH26171. sih.gov.in/sih2026PS (accessed 17 September 2026).
- [2] Qwen Team, Alibaba Group. *Qwen2.5-VL Technical Report*. arXiv:2502.13923, 2025.
- [3] Presidio: an open-source framework for detecting, redacting and anonymising PII. Originally developed at Microsoft, now community-governed under the Data Privacy Stack organisation. github.com/microsoft/presidio
- [4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz. *Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79-90. arXiv:2302.12173.
- [5] W3C. *WebGPU*. W3C Working Draft. w3.org/TR/webgpu/
- [6] Microsoft. *ONNX Runtime Web*. onnxruntime.ai/docs/tutorials/web/
- [7] Hugging Face. *Transformers.js: state-of-the-art machine learning for the web*. huggingface.co/docs/transformers.js
- [8] Ollama: run open-weight language models locally. ollama.com
- [9] J. A. Hanley and A. Lippman-Hand. *If nothing goes wrong, is everything all right? Interpreting zero numerators*. JAMA, 249(13):1743-1745, 1983. doi:10.1001/jama.1983.03330370053031.
- [10] J. Verhoeff. *Error Detecting Decimal Codes*. Mathematical Centre Tract 29, Mathematisch Centrum, Amsterdam, 1969. (The Aadhaar check digit.)

[11] H. P. Luhn. *Computer for Verifying Numbers*. US Patent 2,950,048, 1960. (The payment-card check digit.)

Appendix: Getting it and checking it

Every number in this report can be reproduced from the repository, and nothing below needs an API key or a network connection to us. The repository is private for the moment, while the team is still working in it. We would be glad to add you: tell us the GitHub username to add, or ask and we will open it.

```
github.com/AavaranAI/Aavaran
```

What you need. Node 22 or later and Python 3 for the first two commands; `setup.sh` checks for both and says what is missing rather than failing obscurely. The third also needs Chrome, Firefox, and Ollama with the 7B model, which is roughly a 6 GB download. Only that third command touches the network, and only to fetch the model.

```
git clone https://github.com/AavaranAI/Aavaran.git
cd Aavaran

./setup.sh          # checks tools, installs deps, builds the extension
./test-all.sh      # 17 checks, about a minute, no browser needed
./test-all.sh --full # 24 checks: real browsers and the live model
```

`test-all.sh` prints the scorecard behind Table 1; `--wild` on the scorecard reproduces the third column. The repository also carries the record this report draws on: `RUNBOOK.md` for the demonstration, a findings note for each measurement spike with the numbers that produced it, and the scorecard's own description of its known gaps. We are glad to walk anyone at the institute through it in person instead.